

STM32 CAN 总线

bxCAN 控制器 完全讲解

芯片平台	STM32F407 (Cortex-M4, 168MHz)
CAN 控制器	bxCAN (CAN1 主 + CAN2 从, 共享 28 个滤波器)
协议标准	CAN 2.0A (11位标准帧) / 2.0B (29位扩展帧)
HAL 库版本	STM32 HAL (CubeMX 生成框架)
内容涵盖	协议 / 位时序 / 滤波器 / 收发 / 中断 / 错误处理 / 实战

2026-06-14

第一章 CAN 总线概述

1.1 什么是 CAN 总线?

CAN (Controller Area Network, 控制器局域网) 是由德国 BOSCH 公司于 1986 年为汽车应用开发的串行通信总线，现已成为 ISO 11898 国际标准，广泛用于汽车电子、工业控制、医疗设备和电力系统。

CAN 是一种多主 (Multi-Master)、差分信号、基于消息 (而非地址) 的串行总线，具有高可靠性、强抗干扰能力和完善的错误检测机制。

1.2 CAN 总线的核心特点

特点	说明
多主结构	任意节点都可主动发送，无主从之分，通过仲裁决定优先级
差分信号	CAN_H 与 CAN_L 两线差分传输，抗共模干扰能力强
基于消息	数据帧用标识符 (ID) 标记消息内容，而非目标地址，天然支持广播
非破坏性仲裁	多节点同时发送时，ID 小 (优先级高) 的报文胜出且不被破坏
错误检测	CRC校验、位填充、帧格式检查、ACK校验等5重错误检测机制
速率与距离	最高 1Mbps (40m以内) ， 降速可达 5km (5kbps)
报文长度	每帧数据场最多 8 字节 (CAN FD 可达 64 字节)

1.3 CAN 物理层：差分信号

CAN 使用 CAN_H 和 CAN_L 两根线传输差分电平，定义两种逻辑状态：

逻辑状态	CAN_H 电平	CAN_L 电平	差分电压 (H-L)	总线含义
隐性 (Recessive) 逻辑 1	约 2.5V	约 2.5V	约 0V	总线空闲态
显性 (Dominant) 逻辑 0	约 3.5V	约 1.5V	约 2V	占用总线

★ 显性 (0) 会覆盖隐性 (1) ——这是 CAN 总线仲裁的物理基础。
多个节点同时发送时，只要有一个发显性 0，总线即为 0。

□ CAN 总线两端必须各接一个 120Ω 终端电阻 (共并联为 60Ω) ， 用于消除信号反射。缺少终端电阻会导致通信不稳定甚至完全失败。

1.4 CAN 与其他总线对比

对比项	CAN	UART	SPI	I2C
-----	-----	------	-----	-----

拓扑	多主总线	点对点	一主多从	多主多从
线数	2 (差分)	2 (TX/RX)	4+	2 (SDA/SCL)
速率	≤1Mbps	可变	数十Mbps	≤3.4Mbps
距离	可达数km	短	板级	板级
抗干扰	极强 (差分)	弱	弱	弱
错误检测	完善	校验位	无	ACK

第二章 CAN 协议帧结构

2.1 四种帧类型

帧类型	作用
数据帧 (Data Frame)	发送节点向总线传输数据，最常用
远程帧 (Remote Frame)	请求其他节点发送指定 ID 的数据帧 (RTR=1，无数据场)
错误帧 (Error Frame)	检测到错误的节点主动发送，通知全网错误发生
过载帧 (Overload Frame)	接收方未准备好时发送，请求延迟下一帧

2.2 数据帧的组成 (标准帧 CAN 2.0A)

一个标准数据帧从总线空闲后依次由以下场组成：

字段	位数	说明
帧起始 SOF	1 位	Start of Frame, 1 个显性位，标志帧开始
仲裁场 ID	11 位	报文标识符，决定优先级，ID 越小优先级越高
RTR	1 位	远程发送请求：0=数据帧，1=远程帧
控制场 IDE/r0/DLC	6 位	IDE=0标准帧；DLC=数据长度(0~8字节)
数据场 DATA	0~64 位	实际数据，0~8 字节
CRC 场	16 位	15位CRC + 1位CRC界定符，校验数据完整性
ACK 场	2 位	应答位 + 界定符，接收方正确收到则拉显性
帧结束 EOF	7 位	End of Frame, 7 个连续隐性位

2.3 标准帧 vs 扩展帧

对比项	标准帧 (CAN 2.0A)	扩展帧 (CAN 2.0B)
标识符长度	11 位 (最多 2048 个 ID)	29 位 (11位基本ID + 18位扩展ID)
IDE 位	0 (显性)	1 (隐性)
适用场景	节点少的简单网络	汽车 OBD/J1939 等大型网络
兼容性	两者可在同一总线共存	扩展帧 ID 整体优先级低于标准帧

★ STM32 bxCAN 同时支持标准帧和扩展帧，通过邮箱/滤波器的 IDE 位区分。

2.4 位填充 (Bit Stuffing)

CAN 协议规定：发送方每发送 5 个连续相同电平的位，就自动插入 1 个相反电平的位（位填充），接收方自动删除这些填充位。

目的：保证总线上有足够的电平跳变，使接收方能持续同步时钟（CAN 无独立时钟线）。

位填充作用范围：SOF、仲裁场、控制场、数据场、CRC序列。CRC界定符之后不再填充。

□ 位填充错误（连续6个相同电平）是 CAN 的一种错误类型，会触发错误帧。

第三章 非破坏性仲裁机制

3.1 仲裁原理

当多个节点同时开始发送时，CAN 通过逐位仲裁（按位比较仲裁场 ID）决定谁能继续占用总线，这个过程不破坏获胜报文，称为非破坏性仲裁（CSMA/CD + AMP）。

仲裁规则：每个节点发送 ID 的同时监听总线电平。由于显性（0）覆盖隐性（1），当某节点发送隐性 1 却监听到显性 0 时，说明有更高优先级的报文在发送，该节点立即退出仲裁、转为接收，待总线空闲后重发。

3.2 仲裁过程举例

假设三个节点同时发送，标识符（从高位到低位）如下表，总线最终电平为各节点逐位「线与」的结果（有0则为0）：

位序(高→低)	节点 A (ID=0x6C3)	节点 B (ID=0x6D2)	节点 C (ID=0x6C1)	总线电平	仲裁结果
bit10	1	1	1	1	继续
bit9	1	1	1	1	继续
bit8	0	0	0	0	继续
bit7	1	1	1	1	继续
bit6	1	1	1	1	继续
bit5	0	1←退	0	0	B 退出（发1见0）
bit4	0	-	0	0	A、C 继续
...	...	-	...	...	...
bit1	1	-	0←	0	A 退出，C 获胜

★ ID 数值越小，仲裁场中显性位越多，优先级越高。
工程上把紧急报文（如刹车）分配小 ID，保证其总能优先发送。

3.3 仲裁的关键特性

- 1. 非破坏性：获胜报文不受影响，继续完整传输，总线利用率高。
- 2. 实时性：高优先级报文延迟可预测（最坏情况可计算）。
- 3. 公平性：低优先级报文在总线空闲时仍有机会发送，不会被永久饿死（除非总线持续繁忙）。

第四章 STM32 bxCAN 控制器架构

4.1 bxCAN 简介

STM32F407 内置 bxCAN (basic extended CAN) 控制器，支持 CAN 2.0A/B，最高 1Mbps。F407 有两个 bxCAN：CAN1 (主) 和 CAN2 (从)。

CAN2 没有独立的滤波器组，CAN1 和 CAN2 共享 28 个滤波器组 (编号 0~27)，通过 CAN_FMR 寄存器的 CAN2SB 位划分两者各自占用的滤波器范围。

□ 使用 CAN2 时，必须先使能 CAN1 的时钟 (即使不用 CAN1)，因为 CAN2 的滤波器和时钟依赖 CAN1。这是常见的「CAN2 不工作」陷阱。

4.2 bxCAN 内部结构

发送部分：3 个发送邮箱 (Tx Mailbox 0/1/2)
待发送报文放入空闲邮箱，由发送调度器按优先级 (ID 或邮箱号) 发送
接收部分：2 个接收 FIFO (FIFO0、FIFO1)，每个 FIFO 深度 3 级
通过滤波器决定报文进入哪个 FIFO；FIFO 满时可配置覆盖或丢弃新报文
滤波器部分：28 个滤波器组 (CAN1/CAN2 共享)
每组可配置为屏蔽模式或列表模式，16 位或 32 位宽度

4.3 bxCAN 三种工作模式

模式	说明	用途
正常模式 (Normal)	正常收发，参与总线仲裁和应答	实际通信
静默模式 (Silent)	只接收不发送 (不发显性位)，只监听总线	总线监控/分析
回环模式 (Loopback)	自发自收，发送的数据直接回到自己的接收FIFO，不影响总线	单板自测
回环静默 (Loopback+ Silent)	回环且与总线物理隔离	完全脱离总线的自测

★ 回环模式 (Loopback) 非常适合在只有一块板子、没有第二个 CAN 节点时，验证 CAN 收发代码逻辑是否正确。

第五章 位时序与波特率配置

5.1 CAN 位时序组成

CAN 的一个数据位 (Bit) 被划分为若干个时间份额 (Tq, Time Quantum) , 分为四段:

段	名称	作用	时长
SS	同步段 (SYNC_SEG)	用于节点同步, 跳变沿应落在此段	固定 1 Tq
PTS	传播时间段	补偿网络物理传播延迟	1~8 Tq (含在BS1)
PBS1	相位缓冲段1 (BS1)	补偿边沿误差, 采样点之前	1~16 Tq
PBS2	相位缓冲段2 (BS2)	补偿边沿误差, 采样点之后	1~8 Tq

采样点 (Sample Point) 位于 BS1 结束、BS2 开始处, 是读取总线电平判定 0/1 的时刻。STM32 中 BS1 = SYNC_SEG + PROP_SEG + PHASE_SEG1, 简化为 BS1 和 BS2 两段配置。

5.2 波特率计算公式

一个位的总时间 = Tq × (1 + BS1 + BS2), 其中 Tq = (BRP+1) / CAN_Clock。

波特率 = CAN_Clock / [(BRP) x (1 + BS1 + BS2)]

其中:

CAN_Clock = APB1 时钟频率 (STM32F407 上 APB1 = 42MHz)

BRP = 分频系数 (HAL: Prescaler)

BS1 = 时间段1 (Tq 个数, 含传播段)

BS2 = 时间段2 (Tq 个数)

1 = 同步段 SYNC_SEG (固定)

采样点位置 = (1 + BS1) / (1 + BS1 + BS2) x 100%

5.3 常用波特率配置 (APB1=42MHz)

波特率	Prescaler	BS1	BS2	采样点	总 Tq
1 Mbps	3	11 Tq	2 Tq	85.7%	14
500 kbps	6	11 Tq	2 Tq	85.7%	14
250 kbps	12	11 Tq	2 Tq	85.7%	14
125 kbps	24	11 Tq	2 Tq	85.7%	14
100 kbps	30	11 Tq	2 Tq	85.7%	14

验证 500kbps: 42MHz / (6 × (1+11+2)) = 42000000 / (6×14) = 500000 = 500kbps □

□ 总线上所有节点的波特率和采样点必须严格一致，否则无法通信。
采样点推荐 75%~87.5%，过高或过低都会降低抗干扰能力。

5.4 HAL 库位时序配置

```
// 500kbps @ APB1=42MHz
hcan1.Instance = CAN1;
hcan1.Init.Prescaler = 6; // BRP
hcan1.Init.Mode = CAN_MODE_NORMAL;
hcan1.Init.SyncJumpWidth = CAN_SJW_1TQ; // 再同步跳转宽度
hcan1.Init.TimeSeg1 = CAN_BS1_11TQ; // BS1 = 11Tq
hcan1.Init.TimeSeg2 = CAN_BS2_2TQ; // BS2 = 2Tq
hcan1.Init.TimeTriggeredMode = DISABLE;
hcan1.Init.AutoBusOff = ENABLE; // 自动总线关闭恢复
hcan1.Init.AutoWakeUp = DISABLE;
hcan1.Init.AutoRetransmission = ENABLE; // 自动重传
hcan1.Init.ReceiveFifoLocked = DISABLE; // FIFO满时覆盖
hcan1.Init.TransmitFifoPriority = DISABLE; // 按ID优先级发送
if (HAL_CAN_Init(&hcan1) != HAL_OK) { Error_Handler(); }
```

第六章 接收滤波器配置

6.1 为什么需要滤波器？

CAN 是广播总线，总线上所有报文每个节点都能收到。若全部接收会占满 CPU 和 FIFO。滤波器让 bxCAN 在硬件层面只接收本节点关心的报文 ID，其余报文直接丢弃，减轻 CPU 负担。

6.2 两种滤波模式

模式	原理	适用场景
屏蔽模式 (Mask Mode)	ID 寄存器 + 屏蔽寄存器。屏蔽位=1表示该ID位必须匹配，=0表示该位不关心（任意值都通过）	接收一段连续/相似 ID 的报文
列表模式 (List Mode)	两个寄存器都作为期望ID，只有完全等于列表中ID的报文才通过	只接收几个精确指定的 ID

6.3 滤波器位宽

每个滤波器组可配置为：

- 32 位：1 组可配 1 个 32位屏蔽 或 2 个 32位列表ID（适合扩展帧 29 位 ID）。
- 16 位：1 组可配 2 个 16位屏蔽 或 4 个 16位列表ID（适合标准帧 11 位 ID）。

6.4 屏蔽模式配置示例（接收 ID 0x100~0x10F）

要接收 ID 高位为 0x100、低 4 位任意的报文，ID=0x100，Mask=0x7F0（低4位不关心）：

```
CAN_FilterTypeDef filter = {0};

// 32位屏蔽模式
filter.FilterActivation = CAN_FILTER_ENABLE;
filter.FilterBank = 0; // 使用滤波器组0
filter.FilterMode = CAN_FILTERMODE_IDMASK; // 屏蔽模式
filter.FilterScale = CAN_FILTERSCALE_32BIT;
filter.FilterFIFOAssignment = CAN_RX_FIFO0; // 匹配报文进 FIFO0

// 标准帧 ID 左移 5 位填入高 16 位 (STD_ID 在 bit[31:21])
filter.FilterIdHigh = (0x100 << 5) >> 16; // ID 高16位
filter.FilterIdLow = ((0x100 << 5) & 0xFFFF); // ID 低16位
filter.FilterMaskIdHigh = (0x7F0 << 5) >> 16; // 屏蔽高16位
filter.FilterMaskIdLow = ((0x7F0 << 5) & 0xFFFF); // 屏蔽低16位

// CAN2 起始滤波器组（双CAN时使用，单CAN填14即可）
filter.SlaveStartFilterBank = 14;

HAL_CAN_ConfigFilter(&hcan1, &filter);
```

★ 接收所有报文（不过滤）最简单做法：Mask 全 0，则任意 ID 都通过。
调试初期常用此法确认能收到数据，再逐步加上过滤条件。

6.5 列表模式配置示例（只收 0x123 和 0x456）

```
CAN_FilterTypeDef filter = {0};
filter.FilterActivation = CAN_FILTER_ENABLE;
filter.FilterBank = 1;
filter.FilterMode = CAN_FILTERMODE_IDLIST; // 列表模式
filter.FilterScale = CAN_FILTERSCALE_16BIT; // 16位，可放4个ID
filter.FilterFIFOAssignment = CAN_RX_FIFO0;

// 16位标准帧ID：ID 左移 5 位
filter.FilterIdHigh = (0x123 << 5); // 列表ID1
filter.FilterIdLow = (0x456 << 5); // 列表ID2
filter.FilterMaskIdHigh = (0x123 << 5); // 列表ID3（重复填）
filter.FilterMaskIdLow = (0x456 << 5); // 列表ID4（重复填）
HAL_CAN_ConfigFilter(&hcan1, &filter);
```

第七章 报文收发与中断

7.1 CAN 启动流程

```
// 1. HAL_CAN_Init 已在 CubeMX 中完成
// 2. 配置滤波器 (见第六章)
can_filter_config();

// 3. 启动 CAN (进入正常模式)
HAL_CAN_Start(&hcan1);

// 4. 使能接收中断 (FIFO0 挂起新报文时触发)
HAL_CAN_ActivateNotification(&hcan1, CAN_IT_RX_FIFO0_MSG_PENDING);
```

7.2 发送报文

```
uint8_t can_send(uint32_t std_id, uint8_t *data, uint8_t len) {
    CAN_TxHeaderTypeDef tx = {0};
    uint32_t tx_mailbox;

    tx.StdId = std_id; // 标准帧ID (11位)
    tx.ExtId = 0;
    tx.IDE = CAN_ID_STD; // 标准帧 (CAN_ID_EXT为扩展帧)
    tx.RTR = CAN_RTR_DATA; // 数据帧 (CAN_RTR_REMOTE为远程帧)
    tx.DLC = len; // 数据长度 0~8
    tx.TransmitGlobalTime = DISABLE;

    // 等待有空闲发送邮箱
    uint32_t timeout = 0;
    while (HAL_CAN_GetTxMailboxesFreeLevel(&hcan1) == 0) {
        if (++timeout > 100000) return 1; // 超时失败
    }

    if (HAL_CAN_AddTxMessage(&hcan1, &tx, data, &tx_mailbox) != HAL_OK)
        return 1;
    return 0;
}
```

★ HAL_CAN_AddTxMessage 把报文放入空闲邮箱后立即返回（非阻塞），实际发送由 bxCAN 硬件后台完成。3 个邮箱可排队 3 帧待发。

7.3 接收报文（中断方式）

```
// FIFO0 接收中断回调 (HAL 自动调用)
void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan) {
    CAN_RxHeaderTypeDef rx = {0};
    uint8_t rx_data[8];

    if (HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &rx, rx_data) == HAL_OK) {
        if (rx.IDE == CAN_ID_STD) {
            uint32_t id = rx.StdId; // 标准帧ID
            uint8_t len = rx.DLC; // 数据长度
            // 处理 rx_data[0..len-1]
            process_can_frame(id, rx_data, len);
        }
    }
}
```

7.4 CAN 中断类型

中断使能宏	触发条件
CAN_IT_TX_MAILBOX_EMPTY	发送邮箱变空（发送完成）
CAN_IT_RX_FIFO0_MSG_PENDING	FIFO0 收到新报文
CAN_IT_RX_FIFO1_MSG_PENDING	FIFO1 收到新报文
CAN_IT_RX_FIFO0_FULL	FIFO0 满（3级）
CAN_IT_RX_FIFO0_OVERRUN	FIFO0 溢出（报文丢失）
CAN_IT_BUSOFF	节点进入总线关闭状态
CAN_IT_ERROR	发生错误（错误计数变化等）

第八章 错误处理机制

8.1 CAN 的五种错误类型

错误类型	检测方式
位错误 (Bit Error)	发送节点监听总线，发送值与监听值不符（仲裁场除外）
填充错误 (Stuff Error)	检测到连续 6 个相同电平的位（违反位填充规则）
CRC 错误 (CRC Error)	接收方计算的 CRC 与帧中 CRC 不符
格式错误 (Form Error)	固定格式位场（如界定符、EOF）出现非法电平
应答错误 (ACK Error)	发送方在 ACK 槽未检测到任何接收方的显性应答

8.2 错误计数器与节点状态

每个 CAN 节点维护两个错误计数器：发送错误计数器（TEC）和接收错误计数器（REC）。根据计数值，节点在三种状态间转换：

节点状态	进入条件	行为
错误主动 (Error Active)	$TEC \leq 127$ 且 $REC \leq 127$	正常工作，检测到错误时发送「主动错误帧」（显性）
错误被动 (Error Passive)	$TEC > 127$ 或 $REC > 127$	仍可收发，但只能发送「被动错误帧」（隐性），不干扰总线
总线关闭 (Bus Off)	$TEC > 255$	节点脱离总线，不再收发；需满足条件后才能恢复

★ 错误计数机制让故障节点（如硬件损坏一直发错）逐步「自我隔离」，避免一个坏节点拖垮整个总线，这是 CAN 高可靠性的关键设计。

8.3 Bus Off 自动恢复

节点进入 Bus Off 后，若使能了自动恢复（AutoBusOff=ENABLE），硬件在检测到总线上 128 次连续 11 个隐性位（即总线持续空闲一段时间）后自动恢复到错误主动状态。

```
// 错误回调中检测 Bus Off
void HAL_CAN_ErrorCallback(CAN_HandleTypeDef *hcan) {
    uint32_t err = HAL_CAN_GetError(hcan);
    if (err & HAL_CAN_ERROR_BOF) {
        // Bus Off: 若未开自动恢复，需手动 Stop+Start
        HAL_CAN_Stop(hcan);
        HAL_CAN_Start(hcan);
        HAL_CAN_ActivateNotification(hcan,
```

```
        CAN_IT_RX_FIFO0_MSG_PENDING);  
    }  
}
```

□ Bus Off 频繁出现通常意味着硬件问题：终端电阻缺失、波特率不匹配、接线错误（H/L 反接）或总线干扰。应先排查硬件而非靠软件反复恢复。

第九章 完整工程示例

9.1 回环模式自测（单板验证）

只有一块开发板时，用回环模式验证 CAN 收发代码：

```
// 1. 初始化时设置回环模式
hcan1.Init.Mode = CAN_MODE_LOOPBACK; // 回环
HAL_CAN_Init(&hcan1);

// 2. 配置接收所有 ID 的滤波器
CAN_FilterTypeDef f = {0};
f.FilterActivation = CAN_FILTER_ENABLE;
f.FilterBank = 0;
f.FilterMode = CAN_FILTERMODE_IDMASK;
f.FilterScale = CAN_FILTERSCALE_32BIT;
f.FilterFIFOAssignment = CAN_RX_FIFO0;
f.FilterIdHigh = 0; f.FilterIdLow = 0;
f.FilterMaskIdHigh = 0; f.FilterMaskIdLow = 0; // 全0=接收所有
f.SlaveStartFilterBank = 14;
HAL_CAN_ConfigFilter(&hcan1, &f);

// 3. 启动 + 使能接收中断
HAL_CAN_Start(&hcan1);
HAL_CAN_ActivateNotification(&hcan1, CAN_IT_RX_FIFO0_MSG_PENDING);

// 4. 发送测试帧，会立即回到自己的 FIFO0
uint8_t data[8] = {0x11,0x22,0x33,0x44,0x55,0x66,0x77,0x88};
can_send(0x123, data, 8);

// → 回调 HAL_CAN_RxFifo0MsgPendingCallback 应收到此帧
```

9.2 两节点通信 + FreeRTOS 任务架构

```
#define CAN_RX_QUEUE_LEN 16

typedef struct {
    uint32_t id;
    uint8_t data[8];
    uint8_t len;
} CanMsg_t;

static QueueHandle_t s_can_rx_queue;
```

```
void can_app_init(void) {
    s_can_rx_queue = xQueueCreate(CAN_RX_QUEUE_LEN, sizeof(CanMsg_t));
    can_filter_config();
    HAL_CAN_Start(&hcan1);
    HAL_CAN_ActivateNotification(&hcan1, CAN_IT_RX_FIFO0_MSG_PENDING);
}

// 中断回调：收到报文塞进队列，交给任务处理
void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan) {
    CAN_RxHeaderTypeDef rx;
    CanMsg_t msg;
    if (HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &rx, msg.data) == HAL_OK) {
        msg.id = rx.StdId;
        msg.len = rx.DLC;
        BaseType_t woken = pdFALSE;
        xQueueSendFromISR(s_can_rx_queue, &msg, &woken);
        portYIELD_FROM_ISR(woken);
    }
}

// CAN 处理任务
void can_task(void *arg) {
    CanMsg_t msg;
    for (;;) {
        if (xQueueReceive(s_can_rx_queue, &msg, portMAX_DELAY)) {
            switch (msg.id) {
                case 0x100: handle_temp(msg.data); break;
                case 0x200: handle_motor(msg.data); break;
                default: break;
            }
        }
    }
}
```

★ CAN 接收中断中只做「取报文+入队列」，复杂业务放到任务里处理，保证中断快速返回——这是 RTOS 下处理 CAN 的标准模式。

第十章 常见问题与调试

10.1 CAN 故障排查表

故障现象	可能原因	排查/解决
完全收发不了	缺少 120Ω 终端电阻; CAN_H/CAN_L 接反	总线两端各加120Ω; 用万用量表空闲时差分电压
能发不能收	滤波器配置错误, 报文被全部过滤	先用 Mask 全 0 接收所有, 确认能收到再加过滤
发送一直失败/超时	总线上无其他节点应答 (ACK 错误); 波特率不对	至少需2个节点; 单板测试改用回环模式
频繁进入 Bus Off	波特率不匹配; 采样点设置不当; 干扰	核对所有节点波特率一致; 检查接线和屏蔽
偶尔丢帧	FIFO 溢出, 中断处理太慢	中断中只入队列; 用双 FIFO 分流
CAN2 完全不工作	未使能 CAN1 时钟	即使不用 CAN1 也要开 __HAL_RCC_CAN1_CLK_ENABLE()
ID 收到的值不对	StdId 移位处理错误	标准帧滤波器 ID 需左移 5 位 (填入寄存器高位)

10.2 调试建议

1. 先回环后实战：单板先用 Loopback 模式验证代码逻辑，再接入真实总线。
2. 用 CAN 分析仪：USB-CAN 分析仪（如 PCAN、CANalyst）可抓总线报文，快速定位是发送问题还是接收问题。
3. 示波器量差分波形：确认 CAN_H/CAN_L 电平、位时序、有无终端电阻反射。
4. 从低波特率开始：先用 125kbps 调通，再逐步提高到目标速率，排除时序边界问题。

★ 工程经验：90% 的 CAN 问题是硬件层面——终端电阻、接线、波特率三件事。代码调通后若仍不通，优先怀疑硬件。

10.3 CAN FD 简介（扩展知识）

CAN FD（CAN with Flexible Data-rate）是 CAN 的升级版：

- 数据场最多 64 字节（传统 CAN 仅 8 字节）。
- 数据场可用更高波特率（仲裁段低速保证兼容，数据段高速提升吞吐）。
- STM32F407 的 bxCAN 不支持 CAN FD；STM32G4/H7 系列的 FDCAN 控制器才支持。

□ STM32F407 只有传统 bxCAN，不支持 CAN FD。若项目需要 CAN FD，应选用 STM32G4/H7 等带 FDCAN 外设的芯片。